# Unit 2: Refactored Code

When I first looked at the initial code, it was classic "spaghetti" design. The Order class was trying to be a cart, a calculator, and a cashier all at once. If the Duka (small shop) wanted to add a new payment method, I would have had to rip open the existing code and risk breaking it.

Here is how I fixed it, step-by-step.

```
from abc import ABC, abstractmethod

# ========================================
# 1. Single Responsibility Principle
# ========================================

class Order:
    """
    Manages the shopping cart items and calculates totals.
    It no longer handles payment processing directly.
    """
    def __init__(self):
        self.items = []

    def add_item(self, item_name: str, price: float):
        """Adds a product item to the shopping cart."""
        self.items.append({'name': item_name, 'price': price})

    def calculate_total(self, discount_strategy: 'DiscountStrategy' = None) ->
float:
        """Calculates the total cost, optionally applying a discount strategy."""
        total = sum(item['price'] for item in self.items)
        if discount_strategy:
            total = discount_strategy.apply_discount(total)
        return total


# ========================================
# 2. Open/Closed & Dependency Inversion
# ========================================

class PaymentMethod(ABC):
    """
    Abstract Base Class acting as an interface for payment methods.
    High-level Order flows depend on this abstraction rather than concrete
details.
    """
    @abstractmethod
    def pay(self, amount: float) -> None:
        """Abstract method to process payment."""
        pass


class DiscountStrategy(ABC):
    """
    Abstract Base Class for discount rules.
    Allows new discount types to be added without modifying existing Order logic.
    """
```

```
    @abstractmethod
    def apply_discount(self, amount: float) -> float:
        pass


# ========================================
# 3. Liskov Substitution & Interface Segregation (LSP / ISP)
# ========================================

class CreditCardPayment(PaymentMethod):
    """Processes traditional credit card payments seamlessly."""
    def pay(self, amount: float) -> None:
        print(f"Processing credit card payment of KSh {amount:,.2f}...")


class MpesaPayment(PaymentMethod):
    """Processes local M-Pesa payments seamlessly."""
    def pay(self, amount: float) -> None:
        print(f"Processing M-Pesa payment of KSh {amount:,.2f} via Sim Toolkit/STK
Push...")


class StandardDiscount(DiscountStrategy):
    """Applies a standard percentage-based discount (e.g., 10% off)."""
    def __init__(self, percentage: float):
        self.percentage = percentage

    def apply_discount(self, amount: float) -> float:
        # Deduct the percentage from the total amount
        return amount * (1 - (self.percentage / 100))


# ========================================
# 4. Full Execution Flow & Usage Example
# ========================================
if __name__ == "__main__":
    print("--- Starting Duka Shopping System Simulation --- \n")

    # Step 1: Initialize the customer's shopping cart
    my_order = Order()

    # Step 2: Add local Kenyan items to the cart
    my_order.add_item("Pack of Kericho Gold Tea", 250.00)
    my_order.add_item("2KG Maize Meal", 180.00)
    my_order.add_item("1L Ilara Milk", 110.00)

    print("Items added to cart successfully.")

    # Step 3: Instantiate and apply a 10% Duka discount
    duka_discount = StandardDiscount(percentage=10.0)
    final_total = my_order.calculate_total(discount_strategy=duka_discount)

    print(f"Total calculated with a 10% discount: KSh {final_total:,.2f}")

    # Step 4: Choose a payment method and pay
    # We can easily swap out MpesaPayment for CreditCardPayment here (LSP)
    payment_processor = MpesaPayment()
    payment_processor.pay(final_total)
```

```
print("\n--- Transaction Completed Successfully ---")
```

## How I Applied the Principles

- **SRP:** I stripped payment logic out of `Order`. Now, `Order` only tracks items, while dedicated classes handle payments.
- **OCP:** If my Duka decides to accept a new currency, I don't touch the existing code. I just write a new class that inherits from `PaymentMethod.`
- **LSP:** Both `CreditCardPayment` and `MpesaPayment` implement the `pay()` method seamlessly. I can swap one for the other without crashing the app.
- **ISP:** By keeping my `PaymentMethod` interface strictly focused on the `pay` action, I ensured no subclass is forced to implement dead code.
- **DIP:** The high-level process doesn't care *how* Mpesa or Credit Cards work; it just relies on the `PaymentMethod` abstraction.

### Reflections & Lessons Learnt

Refactoring this Duka system reminded me that writing code that "just works" isn't enough. The initial code was simple, but it was a dead end for growth.

By applying SOLID principles, I realized that taking a little extra time upfront to build abstractions saves massive headaches down the road. The code is now modular; I can test the Mpesa integration independently of the shopping cart logic. For a small business like a Kenyan Duka (small shop), being able to pivot and add new local payment methods or loyalty discounts quickly is a competitive edge, and clean architecture makes that happen.

By separating out the `StandardDiscount,` I noticed how easy it would be to create a Discount (eg a `BlackFridayDiscount)` class in the future without altering a single line of code inside the `Order` class. The usage flow explicitly demonstrates that the high-level simulator handles combining the objects, leaving our business modules entirely independent and robust against changing requirements.

### References


1. Martin, R. C. (2000). *Design Principles and Design Patterns.* Object Mentor, 1-34.
2. Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software.* Addison-Wesley.
3. Liskov, B. H., & Wing, J. M. (1994). *A behavioral notion of subtyping.* ACM Transactions on Programming Languages and Systems (TOPLAS), 16(6), 1811-1841.
4. Meyer, B. (1997). *Object-Oriented Software Construction* (2nd ed.). Prentice Hall.
5. Freeman, E., Robson, E., Sierra, K., & Bates, B. (2004). *Head First Design Patterns.* O'Reilly Media.